



Web Design Assignment 1

Personal Portfolio Website

A responsive portfolio built with HTML and CSS

Submitted by **Monisha Reddy**

Batch ID

GitHub **github.com/Monisha11166**

Live demo (optional)

HTML5 · CSS3 · Flexbox · Responsive design

1. Objective

The assignment asks for a responsive Personal Portfolio Website built with HTML and CSS. My site introduces me, lists my skills in a table, shows three projects and ends with a contact form. It uses only HTML and CSS, with no JavaScript and no framework such as Bootstrap.

The idea behind the design

The assignment lists the CSS box model as something to use, so I made it the centre of the home section. My portrait sits in the content box, with padding, border and margin drawn around it in the colours browser dev tools use. When you hover over it, the margin and padding grow and the labels change to match. The rest of the page stays quiet so that this is the one memorable thing.

Deliverables

- **Source code:** the portfolio-website folder (HTML, CSS, images).
- **Screenshots:** in section 8 of this report and in the screenshots folder.
- **Live demo (optional):** publish with GitHub Pages (steps in section 9).

2. Requirements checklist

Each requirement from the assignment sheet, and where it appears in my project.

Requirement	Where it is in my project
Home, About Me, Skills, Projects and Contact sections	Five <section> elements with the ids home, about, skills, projects and contact, all linked from the menu.
Headings and paragraphs	One h1 in the Home section, an h2 for each other section, h3 for smaller headings, and paragraphs throughout.
Hyperlinks	Menu links, two buttons, project links (live demo and GitHub), an email link and a Back to top link.
Images	A profile image in the home section and a screenshot for each of the three projects. All have alt text and width and height set.
Lists	An ordered list (How I build a page), unordered lists (Quick facts, project tags, contact details, the menu).
Tables	A skills table with a caption, thead and tbody, and th cells with scope attributes.
Forms	A contact form with text, email, select, textarea and checkbox fields, labels, built-in validation and a submit button.
CSS selectors	Element, class and id selectors, descendant and sibling selectors, pseudo-classes (:hover, :focus-visible, :nth-of-type, :target, :checked) and pseudo-elements (::before).
Colors	CSS variables for the ink, paper and accent colours, plus four colours for the box model layers.
Fonts	Lora for headings and Poppins for text, loaded from Google Fonts with safe fallbacks.
Box model	box-sizing: border-box on every element, and a hero diagram that uses real margin, border and padding.
Flexbox	The header, menu, hero, about section, projects, tags, contact section and footer are all laid out with display: flex.
Responsive design	Flex wrapping plus media queries at 900, 760 and 640 pixels, with a CSS-only mobile menu and a table that stacks on phones.
Hover effects and transitions	Menu links, buttons (they lift), table rows, project images (they zoom), the box model diagram (it grows) and form fields (focus ring).

3. Project structure (Step 1)

portfolio-website/	
index.html	All five sections in one page
css/	
style.css	The external stylesheet
images/	
profile.svg	Profile image (replace with a photo)
favicon.svg	Tab icon
project-*.jpg	Three project screenshots
screenshots/	Desktop, tablet and mobile screenshots
docs/	This report
README.md	How to run and edit the site

Keeping the CSS in one external file and the images in their own folder makes the project easy to find your way around, and it is what Step 6 of the assignment asks for.

4. How I followed the steps

Step	What I did
1. Folder structure	Created the folders shown above.
2. HTML page structure	Wrote the doctype, head (title, description, viewport meta tag, fonts, stylesheet link) and one section for each part of the page.
3. Navigation menu	Built a nav with a list of hyperlinks that jump to each section, and made the header stick to the top of the screen.
4. Profile image and content	Added the portrait in the hero, then wrote the About me text, a short process list and the quick facts.
5. Skills table and contact form	Added a table with six skills and a contact form with labelled fields and validation.
6. External CSS	Linked <code>css/style.css</code> . Colours, fonts and spacing are variables at the top of the file.
7. Flexbox layout	Used <code>display: flex</code> , <code>gap</code> and <code>flex-wrap</code> for every section.
8. Responsive design	Added media queries, a CSS-only mobile menu and a stacked version of the table.
9. Testing	Tested at phone, tablet and desktop widths. Results are in section 7.

5. Key parts of the code

Flexbox layout

The header puts the name on the left and the menu on the right. The menu is a list that becomes a row with one line of flex.

```
.header-inner { display: flex; align-items: center; justify-content: space-between; min-height: 64px; }
.site-nav ul { display: flex; gap: 4px; list-style: none; margin: 0; padding: 0; }
```

Projects use Flexbox too. Every second project flips its direction so the picture switches sides:

```
.project { display: flex; align-items: center; gap: 48px; }
.project:nth-of-type(even) { flex-direction: row-reverse; }
```

The box model diagram

Each layer is a real box. The outer one has padding that acts as the margin ring, the next has a real 6 pixel border, and so on. On hover the padding values change and a transition animates the growth.

```
.bm-margin { padding: 28px; background: var(--l-margin); border: 1.5px dashed #D18B4A; }
.bm-border { border: 6px solid var(--l-border); }
.bm-padding { padding: 20px; background: var(--l-padding); }
.bm { transition: padding .35s ease; }
.boxmodel:hover .bm-margin { padding: 44px; }
```

The labels are pseudo-elements, so they can change with the hover as well: `content: "margin 28"` becomes `content: "margin 44"`.

Mobile menu with no JavaScript

On small screens a hidden checkbox stores whether the menu is open. The label next to it looks like a button, and a sibling selector shows the menu when the box is checked.

```
.nav-toggle:checked ~ .site-nav { display: block; }
.nav-toggle:checked + .nav-burger span::before { transform: translateY(6px) rotate(45deg); }
```

Responsive table

On phones the table rows become small stacked blocks. Each cell has a `data-label` attribute, and CSS prints it in front of the value.

```
@media (max-width: 640px) {
  .skills-table, .skills-table tr, .skills-table td { display: block; }
  .skills-table td::before { content: attr(data-label); }
}
```

Contact form thank-you note

The form sends the visitor to `#thanks`. The note is hidden until that part of the address is active, using the `:target` selector.

```
.form-thanks { display: none; }
.form-thanks:target { display: block; }
```

6. Design decisions

Colour

	Ink	#1E2433	Text and dark areas
	Paper	#F6F7FB	Page background
	Accent blue	#2847C7	Buttons, links, step numbers
	Content blue	#9EC5F2	Box model content layer
	Padding green	#BADFA5	Box model padding layer
	Border yellow	#EBC64A	Box model border layer
	Margin orange	#F5B98A	Box model margin layer

Type

Lora, a serif, is used for headings so they feel personal. Poppins, a clean sans-serif, is used for body text so it is easy to read on screens. Body text is 16 pixels with a line height of 1.7, and text blocks stay under about 70 characters wide.

Layout

Text is left-aligned inside a container that is at most 1100 pixels wide. Projects are shown as alternating rows instead of a grid of identical cards, so each one gets a large picture and room to breathe.

Accessibility

- A "Skip to content" link, and a visible focus ring on every link, button and form field.
- Every image has alt text, every form field has a label, and the table uses header cells with scope.
- Colours have strong contrast against their backgrounds.
- Animations are switched off for visitors who set "reduce motion" on their device.

7. Testing (Step 9)

I ran 36 automated checks in a real browser (Chromium) and all of them passed with no console errors. They cover:

- All five sections exist and every menu link points to a real section.
- The page has no script tags, so it really is HTML and CSS only.
- Every required element type is present: headings, paragraphs, links, images, ordered and unordered lists, a table and a form.
- All images load; the table has six rows with header scopes.
- The box model labels and sizes change on hover.
- The form blocks empty or invalid input and shows the thank-you note when valid.
- There is no sideways scrolling at 375, 768, 1024, 1280 and 1600 pixels wide.
- The phone menu opens and closes, and the table stacks on a phone.

Before submitting I also open the site on my own phone to check it by eye.

8. Screenshots

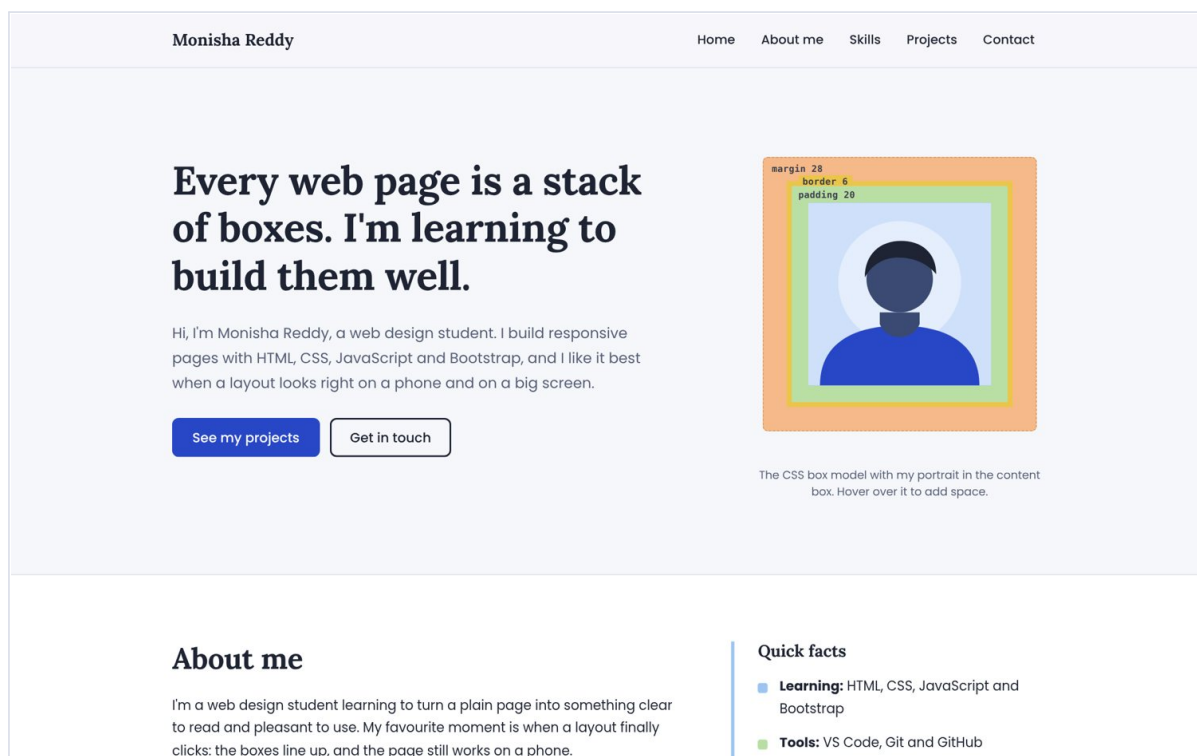


Figure 1. Home section on a desktop screen, with the box model diagram beside the introduction.

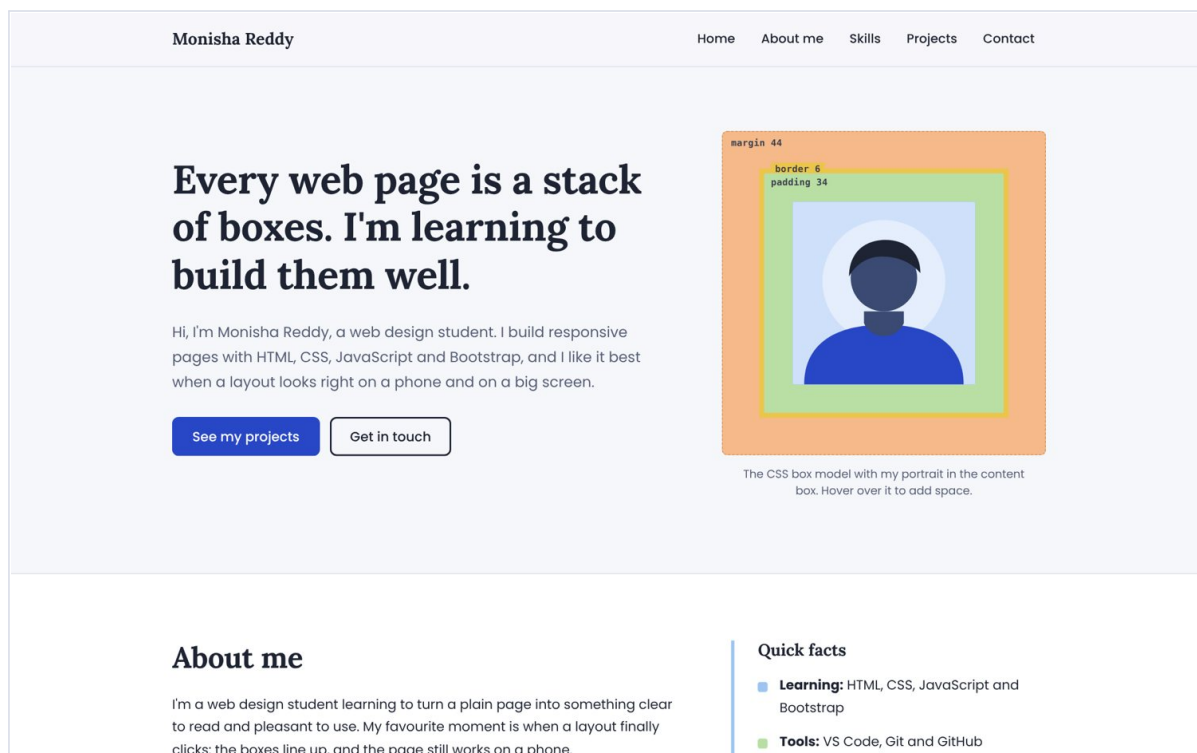


Figure 2. Hovering over the diagram grows the margin and padding, and the labels change to margin 44 and padding 34.

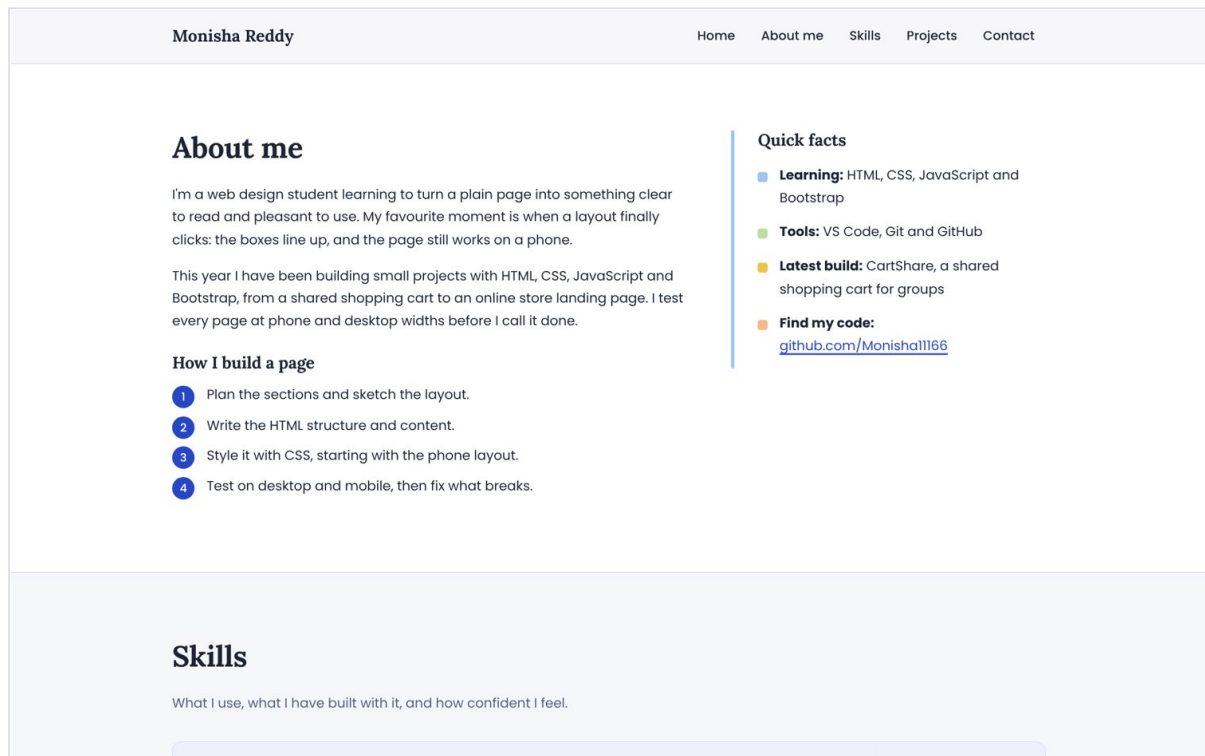


Figure 3. About me, with the process list and the Quick facts list.

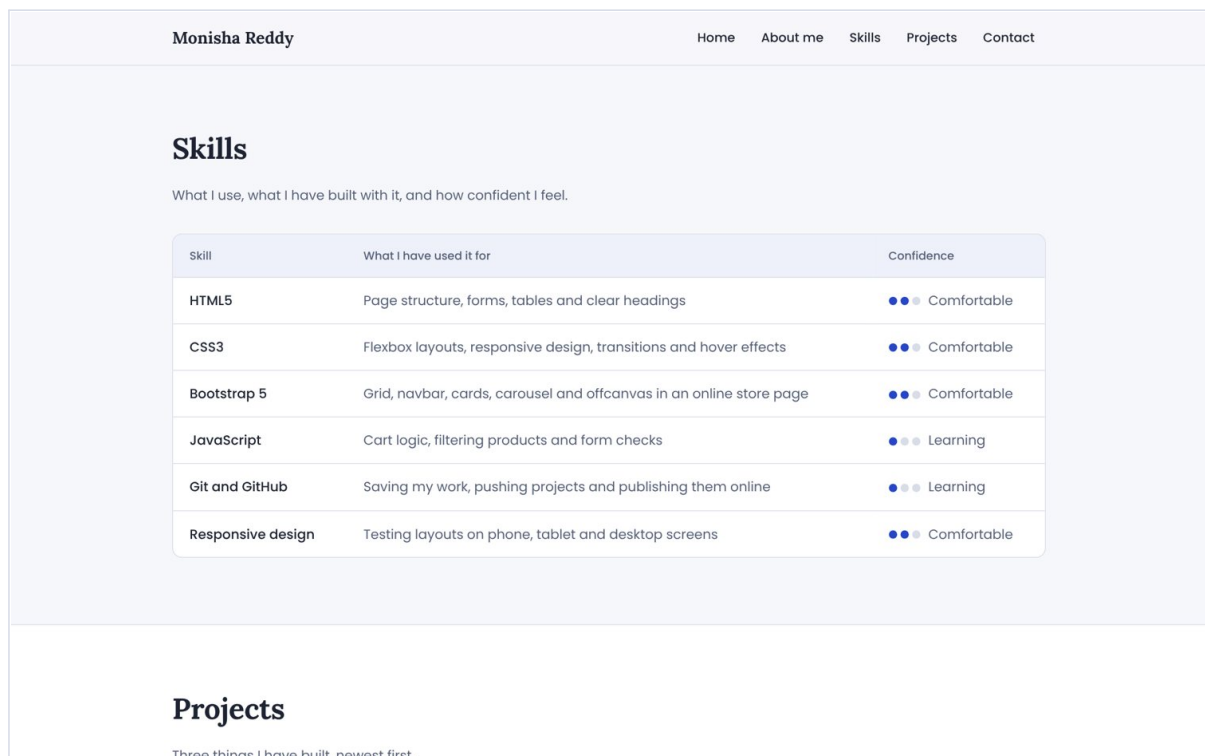


Figure 4. Skills table with confidence dots. Rows highlight on hover.

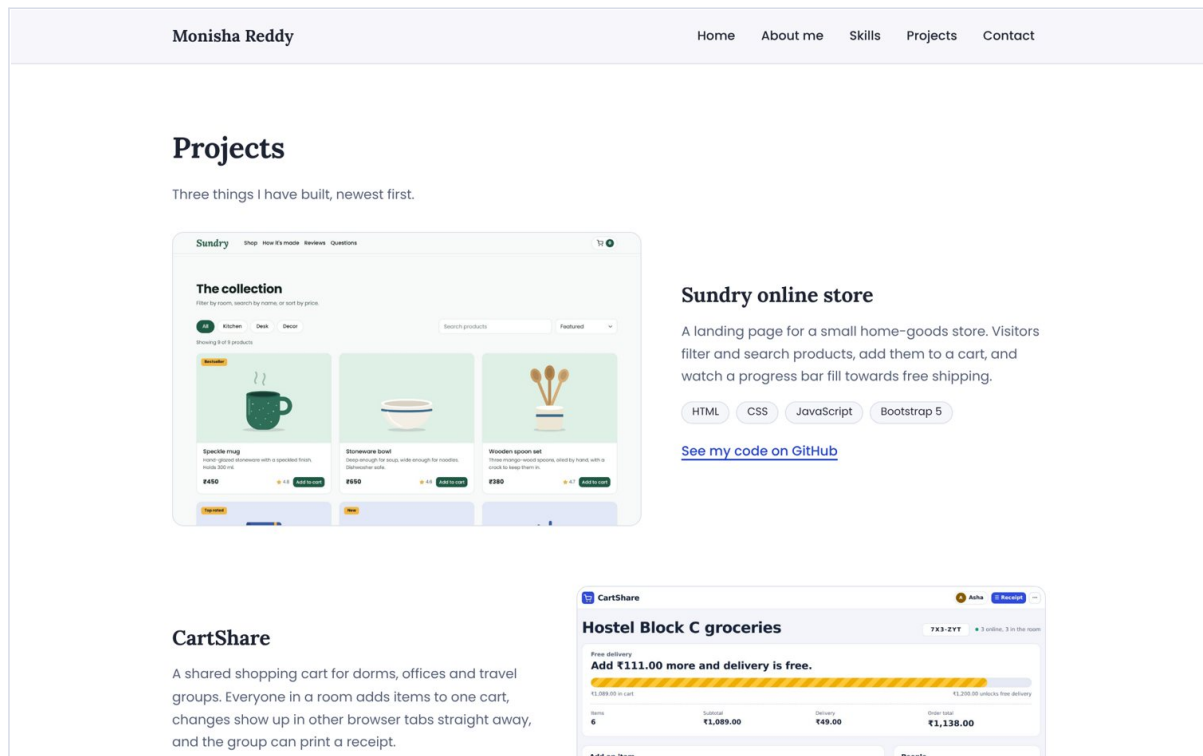


Figure 5. Projects shown as alternating rows with a screenshot, description, tags and links.

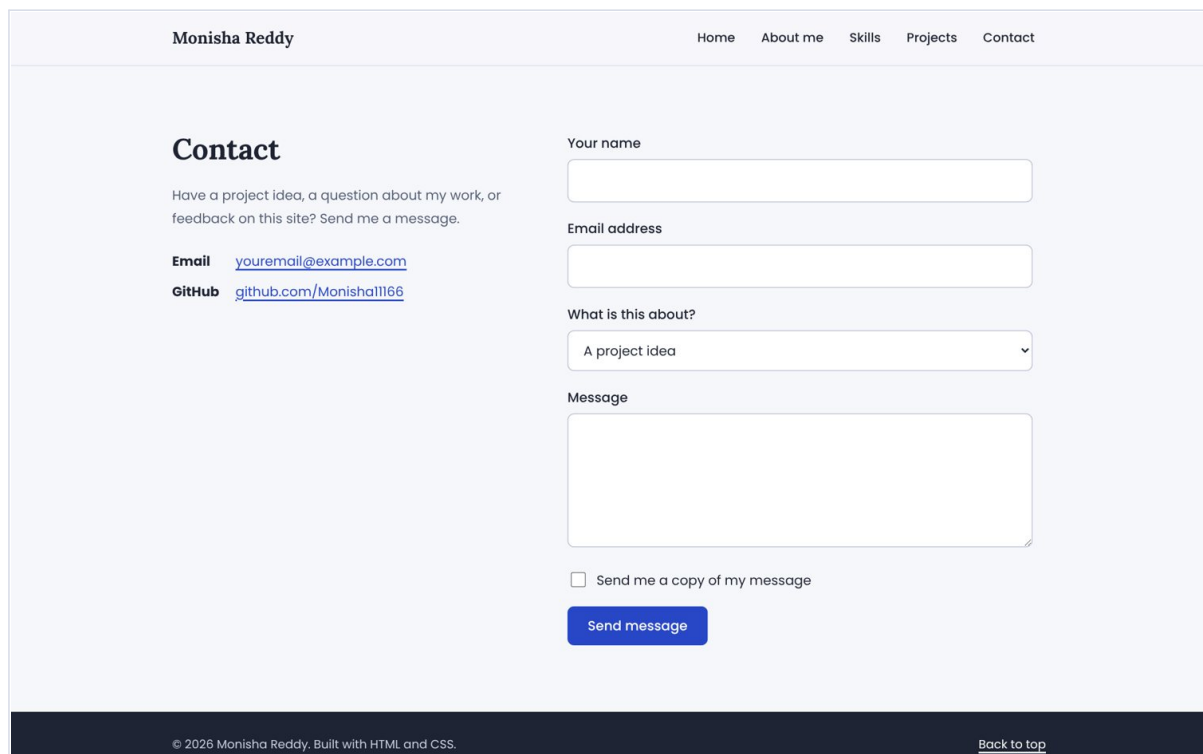


Figure 6. Contact form. Invalid fields are blocked with the browser's built-in messages.

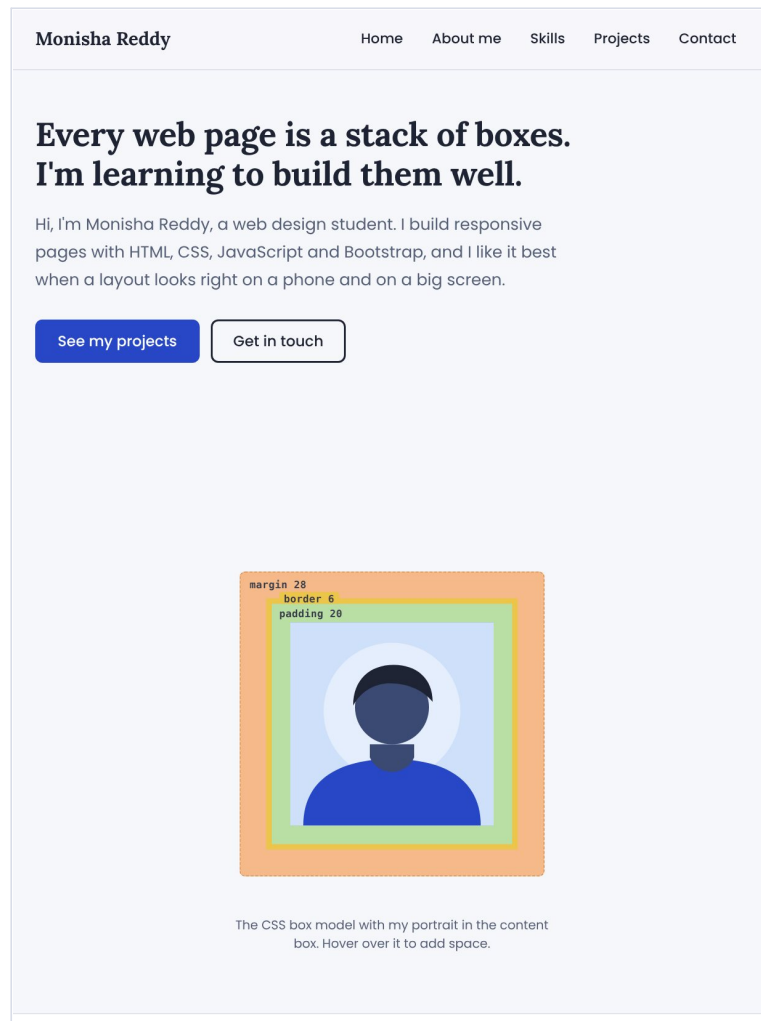


Figure 7. Tablet width (820 px). The layout switches to a single column.

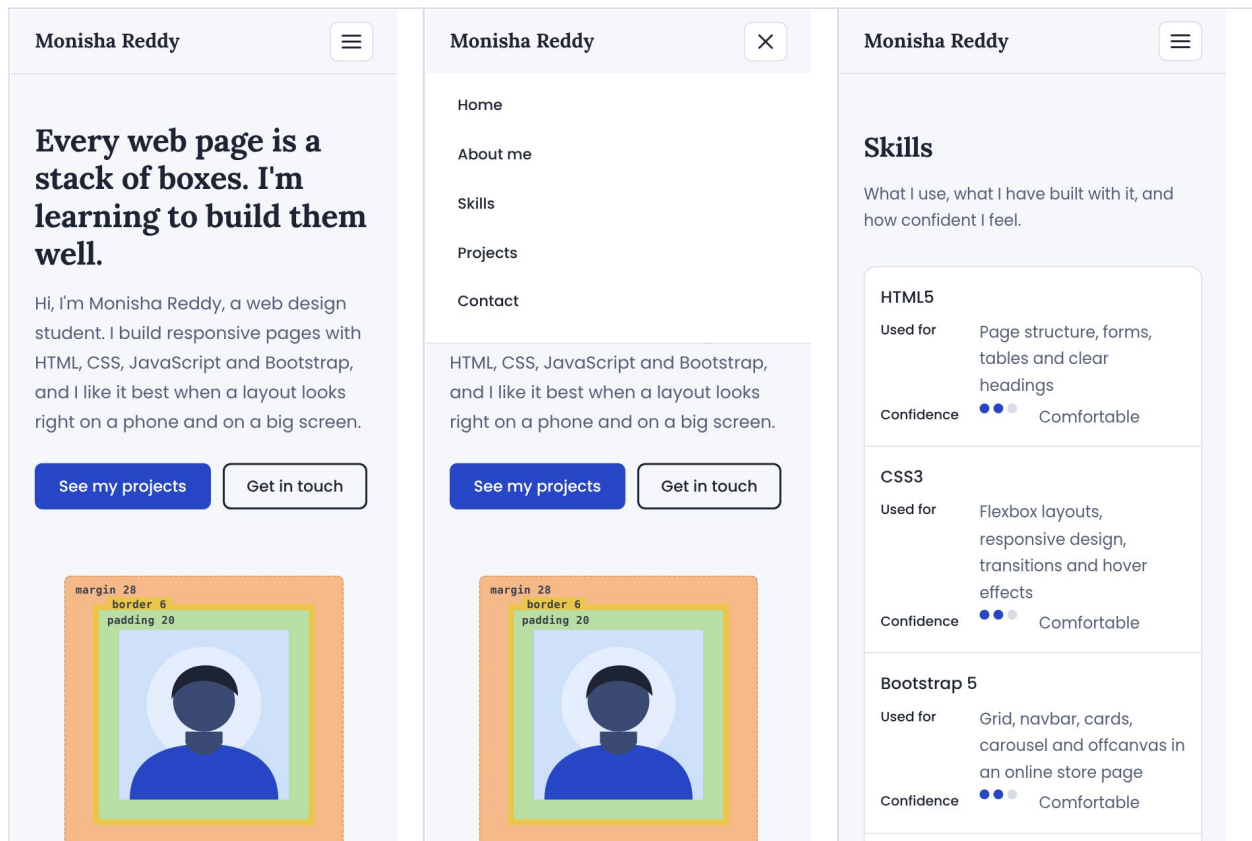


Figure 8. Phone width (390 px): home, the CSS-only menu open, and the skills table stacked into blocks.

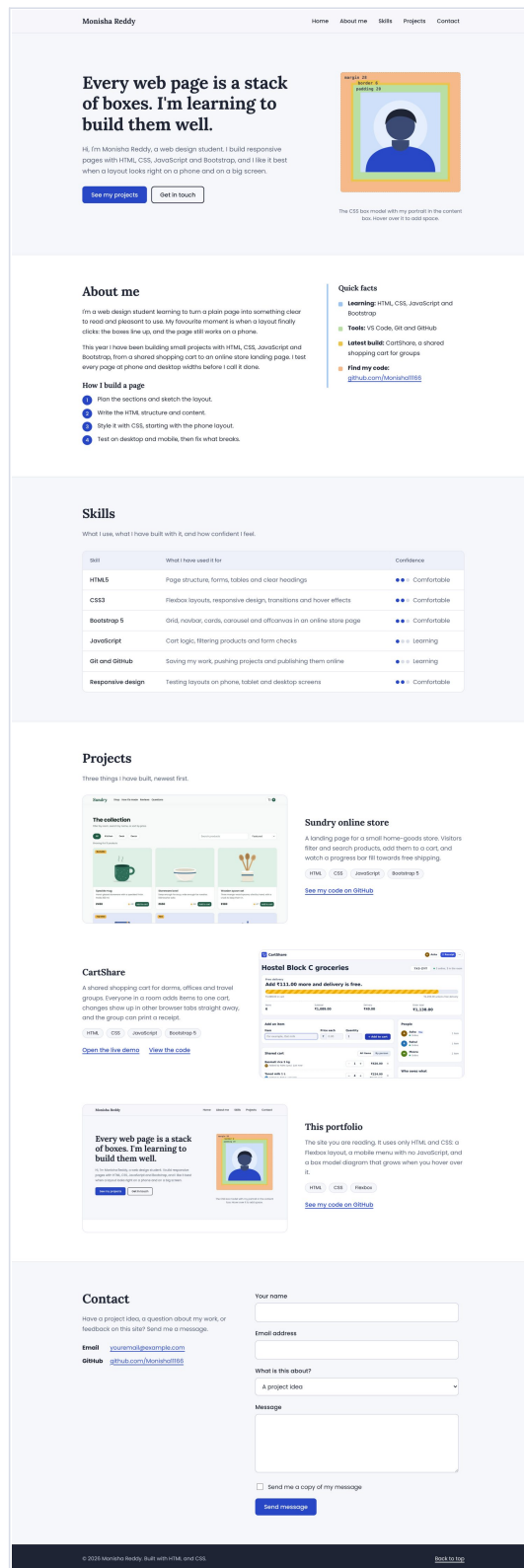


Figure 9. The whole page on a desktop screen.

9. Publishing and submitting

Live demo with GitHub Pages (optional)

- 1 Create a new public repository on GitHub and push the `portfolio-website` folder to it, with `index.html` at the top level.
- 2 Open the repository, then Settings, then Pages.
- 3 Under Source choose "Deploy from a branch", pick main and / (root), and click Save.
- 4 After a minute or two the site is live at `https://your-username.github.io/your-repository/`.

Submitting to Google Drive

- 1 Upload the complete `portfolio-website` folder to Google Drive (it includes the code, the screenshots and this report).
- 2 Right-click the folder, choose Share, and under General access pick "Anyone with the link".
- 3 Make sure the role is Viewer.
- 4 Copy the link and submit it before the deadline. Open the link in a private window to check that it works.

10. What I learned

Building this site showed me how much of layout comes down to the box model: once I understood margin, border and padding, Flexbox and media queries made much more sense. I learned to design for the phone first, to use CSS variables so a colour is changed in one place, and to test at several widths instead of just one. I also found that CSS can do more than I expected without JavaScript, such as the mobile menu, the stacked table and the thank-you note.